

# Building a control-system design simulator: guide for an AI agent

---

**Written from building** `simulator/sample_gas_simulator.html`, an executable spec for a helium purge / O2 feedback controller.

**Aimed at** an agent asked to build the same kind of tool for another loop. **Temperature control** is the worked example here: a heater, a thermal mass, a thermocouple or RTD, and an EPICS `epid` or PLC loop.

**How to use it:** read it before writing code. Most sections are lessons that cost a real iteration with the user.

## 1. What the tool is for

---

- **An executable spec, not the controller.** It exists so a human can *operate* the design before any IOC code is written: press buttons, break hardware, watch alarms, and find design mistakes while they are cheap. Every behaviour you implement must trace to a spec section, and every design change the user makes goes back into the spec.
- **The user will treat it as a design instrument.** They will change requirements while using it: "hand off 0.04 % above target", "the minimum flow should be per lid", "a user run survives a 48 h downtime". The code must absorb that without degrading. Protect it (section 9).
- **Fidelity where it changes decisions, simplicity elsewhere.** Model the plant from data, the actuator from its actual driver, and the control algorithm exactly as the target record implements it. The UI only has to be clear.

## 2. Architecture that survived ~20 rounds of change

---

One self-contained HTML file: no build step, no server, emailable. Plain JavaScript, canvas charts, no libraries.

|            |                                                                                                        |
|------------|--------------------------------------------------------------------------------------------------------|
| Plant      | physics + sensor + actuator + consumable (pure state, <code>step(dt)</code> , 1 Hz sampling)           |
| Controller | state machine + PID record + alarms + accounting (sees ONLY what the real IOC sees)                    |
| Station    | Plant + Controller + history + scenario runner + log; several stations run at once                     |
| UI         | <code>render()</code> at ~12 fps, decoupled from simulation; admin screens generated from field tables |
| Test hooks | <code>window.SIM.run / scenario / selfTest</code> ; screenshot mode via URL hash                       |

Rules that kept it maintainable:

- **Fixed physics step and an integer step counter** ( $t = \text{steps} \times \text{DT}$ ). Never accumulate floating time. The controller ticks on exact integer seconds (`steps % STEPS_PER_S === 0`), and the PID on `sec % scan === 0`. Float drift in tick detection causes skipped or double ticks.
- **The controller must not read plant internals.** Give it the same interface the IOC has: readbacks polled at the device rate, puts that go through the device's own rules.
  - In this project: `Setpoint` writes are silently ignored while the Alicat is on hold, and the totalizer is read, never reset.
  - Everything the controller knows must come through that interface, or the simulator will hide design flaws.
- **Parameters are data.** One defaults object per station for the controller, and a separate one for the plant. Every admin field is a row `{key, label, unit, min, max, step, int}`, from which the input, clamping, tooltip and logging are generated. Adding a parameter is then one line in the defaults plus one row.
- **Keep plant parameters visibly separate** ("simulator only") so nobody mistakes them for controller settings.

- **State entry has side effects; state ticks have conditions.** `enter(state, reason)` performs the writes and logs the transition with a human reason, and each `doState()` only decides when to leave. The reason strings become the operator's "Last action" text and the test oracle.
- **The log is the product.** Every transition, override, alarm raise and clear, and world action goes to one log with severity. Users read it to understand behaviour, and tests grep it.
- **Several independent stations** (two beamline end stations in one IOC) cost almost nothing once Station is a class, and they prove the controller has no hidden globals.

### 3. Implement the real control algorithm, not a textbook PID

The target was the synApps `epid` record. Implement *its* equations and quirks, verified from its source:

- $\text{error} = \text{VAL} - \text{CVAL}$ ;  $P = KP \cdot e$ ;  $\Delta I = KP \cdot KI \cdot e \cdot dt$ ;  $\text{output} = P + I (+ D)$ , clamped to `DRVLL..DRVH`
- anti-windup: I only integrates when the output is inside the limits or moving back inward
- bumpless enable: I is initialised from the output link's current value on `FBON 0 → 1`
- ODEL output deadband
- the sign convention:  $KP < 0$  for a process where more actuator lowers the measurement

For temperature control with a PLC or another record, find that implementation's equations and copy them, including the derivative filter, setpoint weighting and output rate limit.

Controller features that were **added because the simulator exposed a need**, and are likely to transfer to temperature:

| Need found in simulation                        | Mechanism                                                                                                        | Temperature analogue                                                                              |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Loop gain changes with operating point          | Gain scheduling: $KP \times (\text{reference} / \text{setpoint})$ , because the process slope $\propto$ setpoint | Radiation losses grow as $T^4$ , so gain falls at high T. Schedule by setpoint or by heater power |
| Sensor lags the process during fast transients  | Lag-compensated handoff: act on reading $\times \exp(-k \cdot \text{lag})$                                       | Thermocouple lag in a fast ramp: hand off from ramp to hold early, to avoid overshoot             |
| Handoff overshoot                               | Hand off before reaching target (negative $\Delta$ )                                                             | Start the soak before the setpoint, by the measured overshoot                                     |
| Actuator should not chase noise near target     | Near-target "fine band": $KP \times 0.5$ inside an absolute band, with hysteresis                                | The same. Size the band from measured noise, never as a % of setpoint                             |
| Minimum actuator level differs by configuration | Per-configuration output minimum ( <code>DRVLL</code> )                                                          | Minimum heater duty per fixture, or none                                                          |

### 4. Model the plant from the user's data

- **Ask for data early and keep asking.** Every round of real data changed the model:
  - the weekly archive gave volume, leak rates and dead time
  - 10 s data gave lid-lift kinetics and transport delay
  - 1 Hz data gave the noise structure
- **Put raw data and analysis in the repo** with one regenerate-everything script, and make the simulator's parameters cite those results.
- **Validate before using it.** Replay recorded inputs through the model and compare the outputs (here: 99 % of samples within  $\times/\div 1.5$  for one lid). Then check the simulator's statistics against the record, for example Allan deviation and autocorrelation of the sensor output under the same conditions.
- **Model structure for the O2 case:**
  - one well-mixed volume

- a flow-dependent leak per configuration
- a sensor-zone first-order lag
- a variable transport delay (implemented as a history buffer read at  $t - \text{delay}$ , whose delay may only grow at  $< 1$  s/s so readings never run backwards)
- multiplicative seal variability per closure
- smooth blending between regimes (flow-off versus with-flow leak) instead of an if-switch
- **Temperature analogue:**
  - lumped thermal RC:  $C \cdot dT/dt = \eta \cdot P_{\text{heater}} - u \cdot (T - T_{\text{amb}})/R - \epsilon \cdot \sigma \cdot A \cdot (T^4 - T_{\text{amb}}^4)$
  - a sensor lag  $\tau_s$  (thermocouple bead and sheath), and a transport delay if the sensor is remote
  - heater saturation, PWM or SSR quantisation, and slow cooling (no active cooling means asymmetric authority)
  - fixture-to-fixture variation, like the lid seal factor
- **Model the actuator from its real driver.** Read the actual EPICS database and protocol files in production and diff them against upstream; the deployed copy may differ. Reproduce every behaviour that can fool a controller:
  - disabled-while-held records
  - resolution and quantisation
  - built-in ramps
  - readback polling rate
  - totalizers
  - what "Run", "Hold", "Reset" and similar commands actually do

## 5. Noise: measure it, then classify it

---

This project got the noise model wrong twice, and both mistakes changed the controller recommendation.

1. **Fit a structured model, not one white-noise number.**
  - Compute the Allan deviation of a detrended high-rate record.
  - White noise falls as  $1/\sqrt{\tau}$ . A floor or rise shows a slow component. Here that component was  $\sigma$  0.67 m% with  $\tau$  79 s, as an Ornstein–Uhlenbeck process.
  - Fit white + OU to the Allan deviation, and check the autocorrelation. The detrended ACF is biased low at long lags, so use it as a check only.
2. **Classify each component as process or measurement, using the plant's time constants.**
  - A fluctuation faster than the process can move is not process. Here the enclosure mixes on  $V/F \approx 164$  min, so an 80 s wander, or a step, cannot be bulk O<sub>2</sub>.
  - Apply measurement-side noise to the reading only, and score controllers on the **true process variable**, not the reading.
  - Scoring on the reading favoured a tight loop. Scoring on the true O<sub>2</sub> reversed the recommendation.
3. **Pin down vague words.** The user said "the wander is likely real". They meant the *trend*, which had been removed. Ask what a word refers to before encoding it.

For temperature: thermocouple noise is mostly white plus mains pickup, with a cold-junction drift. Real temperature cannot change faster than  $\tau = R \cdot C$  allows.

## 6. Alarms: make them state-aware from the start

---

Naive threshold alarms on a PID loop produce false alarms at every setpoint change. What worked:

- **Debounce:** a level must persist for  $N$  s before an alarm is raised, changed or cleared (the EPICS HYST/delay pattern). Longer persistence (5 min) for slow process alarms.

- **Settling flag:**
  - Set on feedback-on and on every setpoint change, recording the direction.
  - Cleared when the process variable **crosses** the setpoint. "Within band" gave edge blips.
  - While settling, process alarms are held off **only while progress continues**: the slope toward the setpoint, from 20-sample means at each end of a 2 min window, judged after a grace of dead time plus window.
  - A stall latches, and alarms apply at once.
  - A settle timeout catches targets that are never reached.
- **Steadiness gate for actuator-level alarms:** "output too high/low" only when both the output and the process variable are steady. A genuine fault is steady; the PID winding up or down is not.
- **Judge the controller's demand, not the delivered value**, when diagnosing configuration. An empty supply must not look like "wrong mode".
- **Per-alarm classification:** ask the user which conditions matter. "O2 too low" was not an operator alarm, so it became a log note.

Temperature analogues:

- over-temperature trip (hard, not debounced)
- "heater at 100 % for N min while below setpoint" (a heater fault, or setpoint unreachable)
- sensor open or invalid
- "setpoint not reached within X"

## 7. Detect physical faults from kinetics

---

The best feature ideas came from the user's physical insight. Turn them into tests of measured dynamics against the model:

- **Lid open during a purge:**
  - Wait for decay onset (2 % drop), then compare the decay rate over 15 s with F/V.
  - Declare it open if the rate is below half **or** the curvature (2nd-half rate / 1st-half rate) is below 0.8, meaning the decay levels off.
  - Onset-based windows are immune to transport-delay uncertainty.
  - This decided in ~26 s instead of 60, and caught partially open boxes a 10 s window missed.
- **Lid lifted while regulating:** level **AND** rate. Neither alone separates a lift from slow creep.
- **Temperature analogue:**
  - heater disconnected or thermocouple detached: the heating rate after onset is much lower than P/C predicts
  - thermocouple out of the part: the rate is too fast, with no thermal mass
  - door open: the cooling rate is higher than the R-C model

## 8. Accounting and forecasting are cheap and valued

---

- **Consumable run-out without a dedicated sensor:** integrate the actuator's own totalizer (here litres). Fit the usage rate over several windows (3, 2, 1, 0.5, 0.25 days) and show the spread as the confidence range. Alarm on the median. Validate against the actual run-out in simulation.
- **Usage per "run":** define runs from controller events, not calendar time. Here a run is a series of purges, ending at the first flow-zero after the last purge, closed after a gap longer than the longest downtime plus start-up delay. Measure gaps from the event that ended activity, and let an operator mark a changeover.
- **Use exact event values at boundaries.** Interpolating hourly snapshots across a 130 L purge gave a 2 % error.
- **Temperature analogue:** energy per run (kWh), heater hours, element-life forecast.

## 9. Protect the simulator while it evolves

---

- **Tag known-good versions** in git before each substantial change.
- **Built-in self-test:** run every scenario on a hidden station with **default** parameters, and check the end state plus regexes on the log (and custom checks). Show it in the Help page; it runs in ~10 s.
- **Sabotage-test the self-test.** Break the plant deliberately and confirm it fails. This one failed 7 of 19 scenarios with specific reasons.
- **Test headlessly** through `window.SIM` (run N seconds, run a scenario, return state and log) rather than by clicking. Batch parameter sweeps the same way.
- **Scenario resets must reset scenario-relevant settings.** A target changed by one scenario leaked into the next and looked like a controller bug.
- **Watch for off-by-N buffer sizes** when a window length becomes a parameter. The slope-history buffer was 5 samples shorter than the averaged window, so stall detection silently never ran.

## 10. UI lessons

---

- **One source of truth per physical fact.** Two selectors for "which lid" (the controller's mode and the world's lid) produced a baffling "PID won't settle" report. Link them by default, and make the mismatch deliberate and visible.
- Separate the **operator panel** (what the real GUI will show) from the **physical world** (simulator-only controls), and label that separation on screen.
- Three levels (user, admin, deep admin) keep the main panel simple while exposing every number.
- **Time scaling** (1× to 3000×) is essential. Real loops take hours; a 4-day forecast test must run in seconds.
- Graphs need log axes for processes spanning decades, a "true value (sim only)" overlay, the thresholds drawn as lines, and a controller-state strip under the time axis.
- **Tooltips on everything**, and a Help page built from the Markdown docs at build time. A single-file app cannot fetch files over `file://`, so embed them.
- **Screenshot mode** (URL hash → run scenario to time t, open panel) makes illustrated docs reproducible with a headless browser.

## 11. Working with the user

---

- Ask one question at a time, with a recommended option. Record each decision in the spec with its date, and the reason when one was given.
- Report measured numbers, not impressions ("in band after 12 min instead of 19"). When a result surprises you, say so and show the check that explains it.
- When the user's report conflicts with your test, reproduce **their** path, including UI quirks, before defending the code.
- Correct your own misattributions explicitly and everywhere: figure, README, spec, code comments.

## 12. Checklist for a temperature-control version

---

1. Get the heater, sensor and controller hardware list, the EPICS or PLC record type, and the deployed driver files.
2. Get data: a heat-up, a cool-down, a hold at 1 Hz, and an open-door or fault event if possible.
3. Fit the RC model, the sensor lag, the delay and the noise (white + slow). Validate by replay.
4. Implement the plant, the controller (the exact record algorithm), the state machine (IDLE / RAMP / SOAK / HOLD / COOLDOWN / FAULT), the interlocks, and the alarms with settling logic.
5. Add world controls: door, heater fault, sensor fault, supply voltage sag, IOC crash.

6. Add scenarios, one per fault, with expected end states and log patterns. Add the self-test.
7. Build the three-level UI, tooltips, Help and screenshot mode. Tag a version.
8. Iterate with the user. Every change goes to the spec, the simulator and the self-test together.